Inferring Causation from Correlation Statements: A Parser, PC-Algorithm and Classical ML Pipeline on Corr2Cause

Chaitanya Sai Meka, Rakshita, Dhanvin Vadlamudi, Kasula Lalithendra, Jagruthi Pulumati
CSAI Project, Phase 1 (Machine Learning)
Code: https://github.com/Jag2007/AI_ML_Project

***Abstract*—Corr2Cause asks whether a causal claim (e.g. "A directly causes B") must follow from a set of correlation and independence statements written in text. We built a fully reproducible pipeline of five notebooks: data cleaning and leakage removal, a rule-based parser with the PC causal discovery algorithm, TF-IDF plus structured graph features, three tuned classifiers, and a DAG and Bayesian network built from the predictions. Text features alone reach an F1 of 0.47 on validation; adding the graph features raises it to 0.80. The chosen Random Forest reaches a test F1 of 0.905 (95% CI 0.852–0.937), the predicted edges have a precision of 0.89, and a Bayesian network fitted on our DAGs answers all five intervention queries within 0.05 of the true graph.**

I. Introduction

Large language models often treat correlation as causation. Corr2Cause [1] tests this directly: each example is a *premise* that lists, for a closed system of 2–6 variables, which pairs correlate and which are (conditionally) independent, and a *hypothesis* about a causal relation. The label is 1 only if the hypothesis holds in *every* causal graph consistent with the premise. Our goal in Phase 1 is to solve this with classical machine learning and explicit causal reasoning instead of a black box, so every prediction can be traced back to a graph.

II. Literature Review

Causal graphs and interventions are formalised by Pearl [2]. The PC algorithm [3] recovers a causal graph from conditional independences: it removes edges between independent pairs, orients colliders and then applies Meek's rules [4]. The result is a CPDAG, which represents a Markov equivalence class (MEC) of DAGs that share the same skeleton and colliders [5]. Jin et al. [1] built Corr2Cause from exactly this theory and showed that fine-tuned language models fail to generalise on it, and Kıcıman et al. [6] report similar gaps in LLM causal reasoning. For the ML side we use TF-IDF [7], logistic regression, Random Forests [8] and gradient boosting [9] from scikit-learn [10], the bootstrap [11] for confidence intervals, and pgmpy [12] for PC and Bayesian networks. Our contribution is to combine the symbolic PC output with learned models and to measure how much each part helps.

III. Dataset and Quality Checks (Notebook 01)

We use the official Hugging Face splits of Corr2Cause (Table I). Each row has an `input` text, a binary `label`, `num_variables` and a `template` (the hypothesis type).

TABLE I
Splits before and after cleaning

Split	Raw rows	Final rows	Premises
Train (subsampled)	205,734	35,340	400
Validation	1,076	1,026	19
Test	1,162	1,162	21

Inspection. We used `head()`, `tail()`, `shape` and `info()` to look at the structure, `isnull().sum()` to count missing values and `duplicated().sum()` to count duplicate rows. There were no missing values and no duplicate rows in any split, so no imputation or row removal was needed.

Cleaning. Every `input` contains exactly one "Hypothesis:" marker, so we split it into a `premise` and a `hypothesis` column, collapsed repeated whitespace, and dropped the original `input` column. We checked that the number of variables written in each premise matches `num_variables` (0 mismatches). Each premise was given a `premise_id`, because one premise is reused with up to 90 hypotheses.

Leakage. Two premises were shared between train and validation and three between validation and test. We kept the test set untouched and removed the shared premises from validation and train, after which no premise appears in two splits.

Subsampling. Train has 205,734 rows. We sampled 400 whole premises (seed 42), keeping all their hypotheses, which gives 35,340 rows. The share of label 1 is the same before and after (0.185).

IV. Exploratory Data Analysis

Fig. 1 (left) shows that only 13–19% of hypotheses are valid. A model that always says "invalid" would get over 80% accuracy, so we report precision, recall and F1 and use class weights. Fig. 1 (right) shows that the "child" type is never valid, but its text ("X directly causes Y") is identical to "parent". The `template` column would therefore leak the label, so we removed it from the features and used it only for error analysis. Train contains only 5–6 variable graphs, while validation and test also contain 2–4 variable graphs, so the test set is slightly out of distribution. Premises are long (they list every independence) while hypotheses are short and come from six fixed templates.

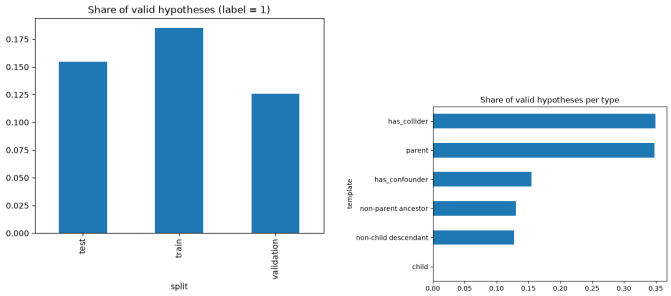


Fig. 1. Left: share of valid hypotheses per split; only 13–19% are valid, so the classes are imbalanced. Right: share of valid hypotheses per type in train; "child" is never valid.

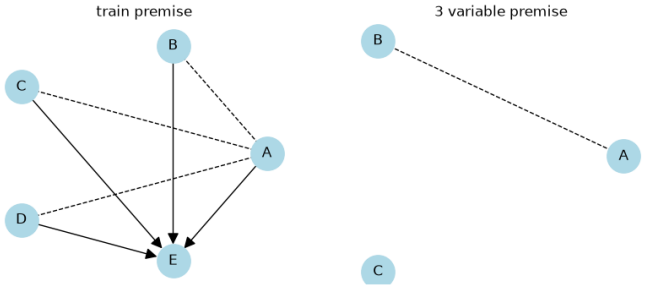


Fig. 2. PC output for two premises. Solid arrows are edges whose direction is certain, dashed lines are edges that can point either way. Left: E is a collider of A–D. Right: only A–B is connected and C is isolated.

V. Parsing and the PC Algorithm (Notebook 02)

Parser. The text is template-generated, so regular expressions are enough. The two variables of a hypothesis are its single capital letters, and its relation (direct cause, indirect cause, collider, confounder) is found from keywords. Schema rules reject variables outside A–F, self-loops (A→A), variables not declared in the premise and unknown wording; all rows passed, and hand-made broken examples were caught.

PC algorithm. PC normally runs statistical tests on numeric data. Here the premise states the independences, so we pass them to pgmpy's PC as an oracle (`ci_test="independence_match"`). PC (i) starts from a complete graph and removes edge $X - Y$ whenever $X \perp Y \mid S$ is stated, (ii) orients $X \to Z \leftarrow Y$ when X, Y are not adjacent and $Z \notin S$, and (iii) applies Meek's rules. Edges it cannot orient stay undirected (Fig. 2).

Equivalence class and support. For every undirected edge we try both directions and keep the orientations that are acyclic and I-equivalent to the PC graph; these are all DAGs in the MEC (1 to 144 per premise). For a hypothesis h we define

$$\text{mec_support}(h) = \frac{1}{|\mathcal{M}|} \sum_{G \in \mathcal{M}} \mathbb{1}[h \text{ holds in } G], \qquad (1)$$

where $\mathcal{M}$ is the MEC. The rule "valid iff mec_support $= 1$" reproduces 93.7% (train), 96.5% (validation) and 96.2% (test) of the labels, which confirms that the parser is correct. The remaining disagreements are cases where the dataset's label differs from PC's conclusion.

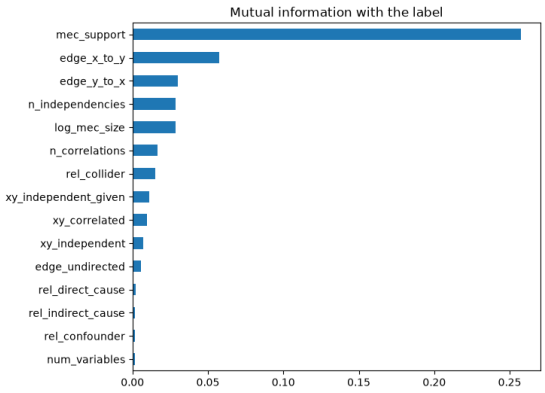


Fig. 3. Mutual information between each structured feature and the label (train). The PC-based `mec_support` is by far the most informative feature.

TABLE II
Feature ablation (logistic regression, validation)

Features	Precision	Recall	F1
TF-IDF only	0.311	0.953	0.469
Structured only	0.634	0.915	0.749
TF-IDF + structured	0.695	0.938	**0.799**

VI. Feature Engineering and Selection (Notebook 03)

Structured features (15). Claim type (one-hot); counts of correlations and independences; whether the premise says X, Y correlate, are independent, or are independent given others; whether PC drew $X \to Y$, $Y \to X$ or an undirected $X - Y$; $\log |\mathcal{M}|$; and `mec_support`. There are no missing values, and the features are standardised with a scaler fitted on train only.

Text features. Separate TF-IDF vectorisers (unigrams and bigrams, min_df $= 2$) for premise (135 terms) and hypothesis (106 terms), fitted on train only. The token pattern keeps one-letter words because A–F are the variable names.

Selection. Chi-squared scores show that the top text terms are claim words ("collider") and variable letters, which carry no reasoning about the specific pair. Mutual information (Fig. 3) ranks `mec_support` far above everything else, followed by the edge-direction features. Keeping only the top 50+30 TF-IDF terms changes validation F1 very little (0.80 vs 0.82), so we keep all terms.

Ablation. Table II compares the feature sets with logistic regression on validation. Text alone predicts almost everything as valid (recall 0.95, precision 0.31); the structured features fix this.

Fig. 4 confirms the same picture from two different models: the graph features, not the words, drive the predictions.

VII. Models and Evaluation (Notebook 04)

Setup. The inputs are TF-IDF + scaled structured features (256 columns), fitted on train only. We tuned three models with `GridSearchCV` and 5-fold `GroupKFold` grouped by

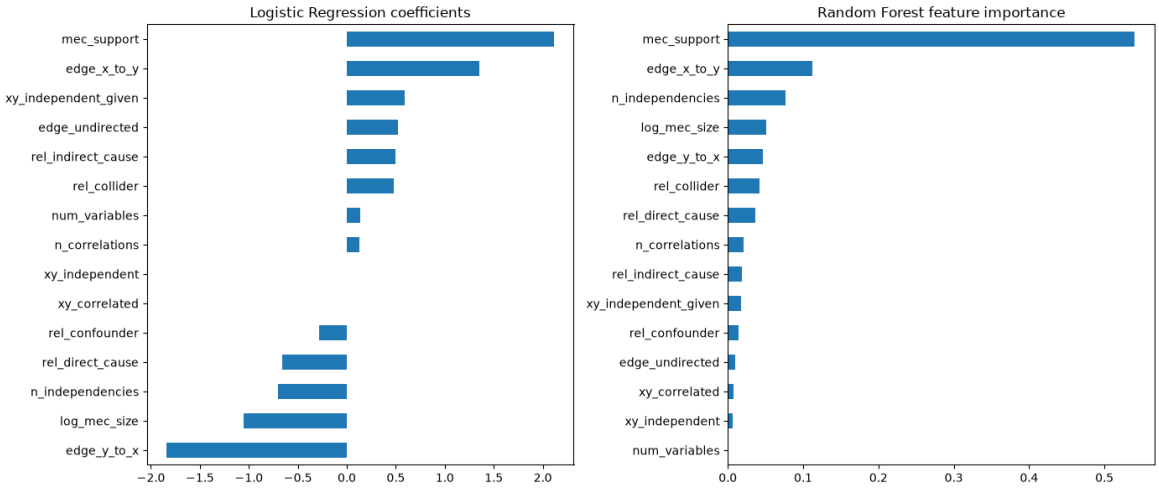


Fig. 4. Left: logistic regression coefficients on the standardised structured features; `mec_support` and `edge_x_to_y` push towards "valid" and `edge_y_to_x` pushes towards "invalid". Right: Random Forest impurity importance, dominated by `mec_support` (0.54).

TABLE III
RESULTS (F1 ON VALIDATION; PRECISION, RECALL, F1 ON TEST)

Model	Val F1	Test P	Test R	Test F1
Logistic Regression	0.784	0.783	0.922	0.847
Random Forest (chosen)	**0.855**	0.860	0.956	**0.905**
Gradient Boosting	0.843	0.907	0.972	0.938
Symbolic rule (no learning)	–	0.920	0.828	0.871

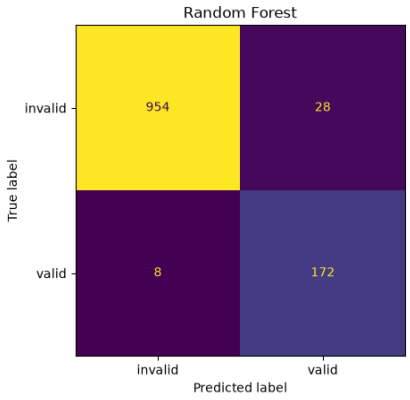


Fig. 5. Test confusion matrix of the Random Forest: 954 true negatives, 28 false positives, 8 false negatives and 172 true positives. Most errors are false alarms.

premise, so all hypotheses of a premise stay in the same fold and the CV score is not inflated. All models use balanced class weights: logistic regression ($C \in \{0.1, 1, 10\}$), Random Forest (300 trees, depth and leaf size) and histogram gradient boosting (learning rate and leaf count). The model is chosen by validation F1, and the test set is used once at the end.

Results. Random Forest had the best validation F1 and was selected (Table III). Gradient boosting scores higher on test, but changing the model after seeing test results would be test-set tuning, so we keep the validation choice. The learned model beats the pure rule (F1 0.871) mainly through higher recall. A premise-level bootstrap (1,000 resamples of whole premises, since rows of one premise are not independent) gives a 95% CI of 0.852–0.937 for the test F1.

Error analysis. Of the 36 errors (Fig. 5), 14 are false alarms on collider claims and 8 on parent claims, the same types where PC and the dataset labels disagree in notebook 02. "Child" claims have no errors. The 2–4 variable graphs, never seen in training, have a similar error rate (4.0%) to the 5–6 variable graphs (3.0%), because the structured features come from the parser and PC rather than from memorised sentences.

VIII. DAG CONSTRUCTION AND BAYESIAN NETWORK (NOTEBOOK 05)

DAGs. Every "X directly causes Y" claim predicted as valid becomes a candidate edge. Edges are added most confident first and are skipped if they are self-loops, use a variable outside the premise, or would create a cycle; all 21 test graphs are valid DAGs. Out of 73 drawn edges, 65 are true, giving an **edge precision of 0.89** and an edge recall of 1.00. All 8 wrong edges are supported by the PC graph (precision 1.00 against PC), so they come from the same label disagreement (Fig. 6).

Bayesian network and interventions. Corr2Cause has no numeric data, so we gave the true graph conditional probability tables, simulated 5,000 samples and fitted our DAG on them with pgmpy (Bayesian estimator, BDeu prior). The causal effect is $P(Y{=}1 \mid do(X{=}1)) - P(Y{=}1 \mid do(X{=}0))$. On the chain A→B→C the true effect of $do(A)$ on C is 0.490 and the correctly oriented graph gives 0.498, while the reversed graph fits the same data but gives 0.000 and wrongly claims that C affects A. On the five test premises whose graph is fully known, all five intervention answers from our network match the true graph within ± 0.05. Correct edge directions

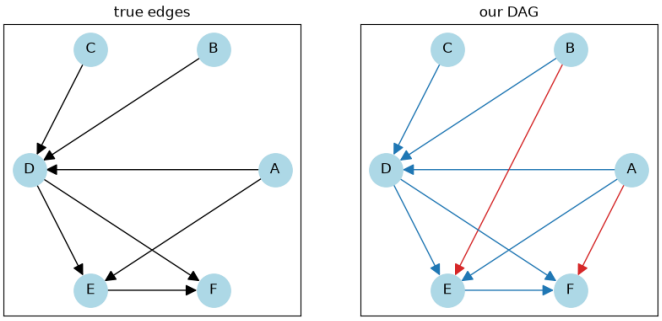


Fig. 6. True edges (left) and our predicted DAG (right) for one test premise. Blue edges are correct; the two red edges (B→E, A→F) are extra edges that PC supports but the dataset labels as invalid.

are therefore what make the causal answers correct.

IX. Conclusion and Phase 2

A parser and the PC algorithm turn text into causal graphs that explain 94–96% of the labels, and combining these graph features with TF-IDF in a Random Forest gives a test F1 of 0.905 with clean, leakage-free splits. The main limitation is the small set of labels that disagree with PC, which accounts for most remaining errors. In Phase 2 we plan to replace TF-IDF with a fine-tuned transformer encoder and combine it with the PC features as a hybrid model, with ablations to show what each part adds.

Contribution Matrix

TABLE IV
Team contributions (✓ = owner of that part)

Task	Chaitanya	Rakshita	Dhanvin	Lalithendra	Jagruthi
Data loading, quality checks	✓				
Cleaning, leakage, subsample	✓				
EDA and plots	✓				
Parser and schema rules		✓			
PC algorithm, MEC support		✓			
Feature engineering			✓		
Feature selection, ablation			✓		
Model tuning and selection				✓	
Evaluation, CI, error analysis				✓	
DAG construction, edge precision					✓
Bayesian network, interventions					✓
Report and repository					✓

References

[1] Z. Jin, J. Liu, Z. Lyu, S. Poff, M. Sachan, R. Mihalcea, M. Diab, and B. Schölkopf, "Can large language models infer causation from correlation?" in *Proc. ICLR*, 2024.

[2] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge Univ. Press, 2009.

[3] P. Spirtes, C. Glymour, and R. Scheines, *Causation, Prediction, and Search*, 2nd ed. MIT Press, 2000.

[4] C. Meek, "Causal inference and causal explanation with background knowledge," in *Proc. UAI*, 1995, pp. 403–410.

[5] T. Verma and J. Pearl, "Equivalence and synthesis of causal models," in *Proc. UAI*, 1990, pp. 255–268.

[6] E. Kıcıman, R. Ness, A. Sharma, and C. Tan, "Causal reasoning and large language models: Opening a new frontier for causality," *arXiv:2305.00050*, 2023.

[7] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[8] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[9] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[11] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[12] A. Ankan and A. Panda, "pgmpy: Probabilistic graphical models using Python," in *Proc. SciPy*, 2015.